

TURNIFY-PRO: Definición del Proyecto

1. Objetivo del Sistema

¿Qué problema resuelve la aplicación? Optimizar el flujo de atención y reducir los tiempos de espera física mediante un sistema inteligente de asignación y monitoreo de turnos en tiempo real, mejorando la experiencia del cliente y la eficiencia operativa del personal.

2. Alcance

¿Qué se entregó finalmente? Para garantizar la efectividad de TURNIFY-PRO, el sistema entregado incluye:

- **Gestión de colas virtuales:** Generación de turnos digitales desde un dispositivo móvil o tótem de entrada.
- **Monitoreo en tiempo real:** Pantallas o tableros de visualización donde los turnos avanzan y se actualizan al instante.
- **Sistema de notificaciones:** Alertas automáticas que avisan al usuario cuando su turno está próximo o ha sido llamado.
- **Métricas básicas de atención:** Registro de tiempos de espera y duración de la atención para medir la eficiencia del servicio.

3. Roles del Sistema

¿Qué puede hacer un ADMIN y qué puede hacer un CLIENTE?

Rol ADMIN (Administrador / Personal de atención):

- Abrir, pausar o cerrar cajas/módulos de atención.
- Llamar al siguiente turno en la fila.
- Reasignar, saltar o cancelar turnos en caso de que el cliente no se presente.
- Visualizar el panel de control con estadísticas en tiempo real (cantidad de personas en espera, tiempos promedio de atención).

Rol CLIENTE (Usuario final):

- Solicitar un turno (escaneando un código QR, a través de un enlace o presencialmente).

- Visualizar su posición en la fila y el tiempo estimado de espera en tiempo real desde su teléfono.
- Recibir alertas cuando faltan pocas personas para su turno.
- Cancelar su turno si decide retirarse, liberando el espacio en el sistema.

2. Arquitectura del Sistema (El "Stack" Tecnológico)

El éxito de **TURNIFY-PRO** radica en una arquitectura híbrida que combina la robustez de un backend tradicional con la agilidad de los servicios en la nube y la comunicación bidireccional en tiempo real.

● Frontend (Cliente)

- **Tecnologías:** Django Templates (HTML5, CSS3, JavaScript).
- **Justificación:** Se optó por el sistema de plantillas de Django para garantizar una **integración nativa y fluida con el backend**. Esto permite una renderización rápida del lado del servidor (SSR), facilitando que el SEO y la carga inicial de la gestión de turnos sean eficientes, mientras que JavaScript maneja la interactividad necesaria para las actualizaciones dinámicas.

● Backend (Servidor)

- **Tecnologías:** Python y Django.
- **Justificación:** Se eligió Django por su principio "*batteries included*", que ofrece **seguridad de alto nivel contra vulnerabilidades** (como inyección SQL o Cross-Site Scripting) y una estructura robusta para manejar la lógica de negocio compleja que requiere la medición de tiempos de atención.

● Base de Datos y Autenticación

- **Tecnología:** Firebase Firestore y Firebase Auth.
- **Justificación:** * **Firestore:** Al ser una base de datos NoSQL orientada a documentos, permite una **sincronización en tiempo real** de los estados de los turnos sin necesidad de recargar la página.
 - **Auth:** Delegar la autenticación a Firebase garantiza un manejo seguro de las sesiones de usuario, permitiendo un inicio de sesión rápido y confiable tanto para clientes como para administradores.

● Almacenamiento en la Nube

- **Tecnología:** Cloudinary.
- **Justificación:** Utilizado para la **gestión optimizada de imágenes de perfil**. Cloudinary permite el escalado y recorte automático de las fotos, reduciendo el consumo de ancho de banda y asegurando que las imágenes se carguen rápidamente en cualquier dispositivo.

● Tiempo Real y Comunicación

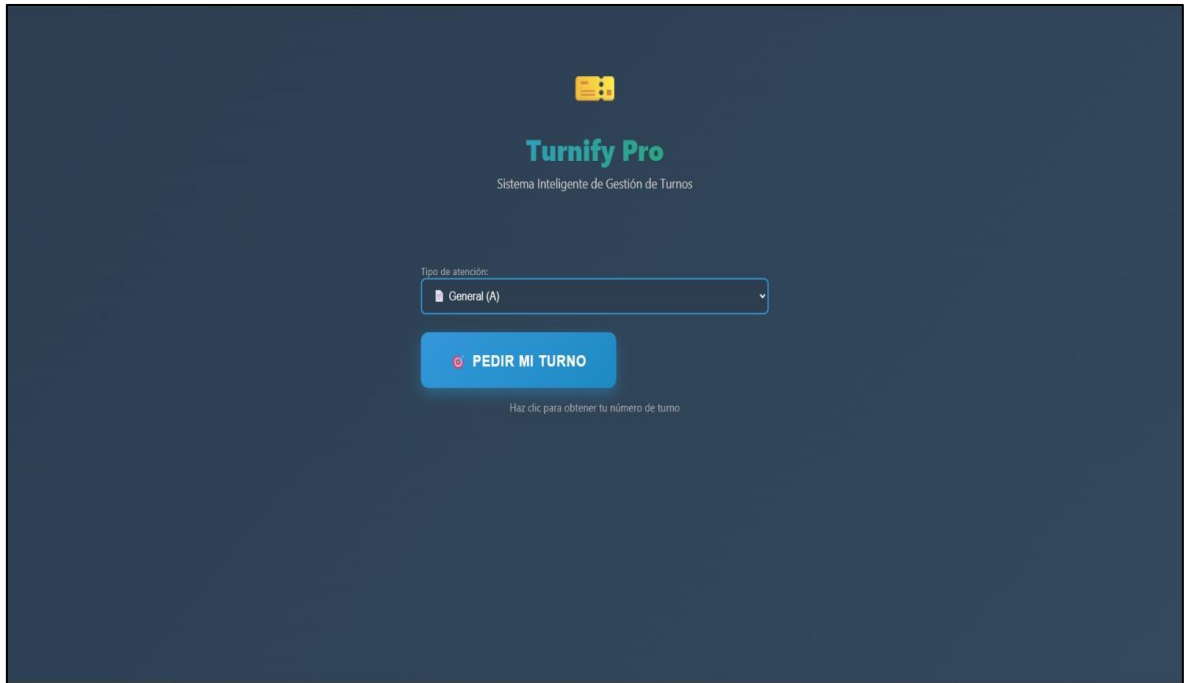
- **Tecnologías:** Django Channels (WebSockets) y Firebase Cloud Messaging (FCM).
- **Justificación:** * **Django Channels (WebSockets):** Es la pieza clave para el chat y el movimiento de turnos. A diferencia de las peticiones HTTP convencionales, los WebSockets

permiten una **comunicación bidireccional abierta**, permitiendo que el administrador "empuje" una actualización al cliente instantáneamente.

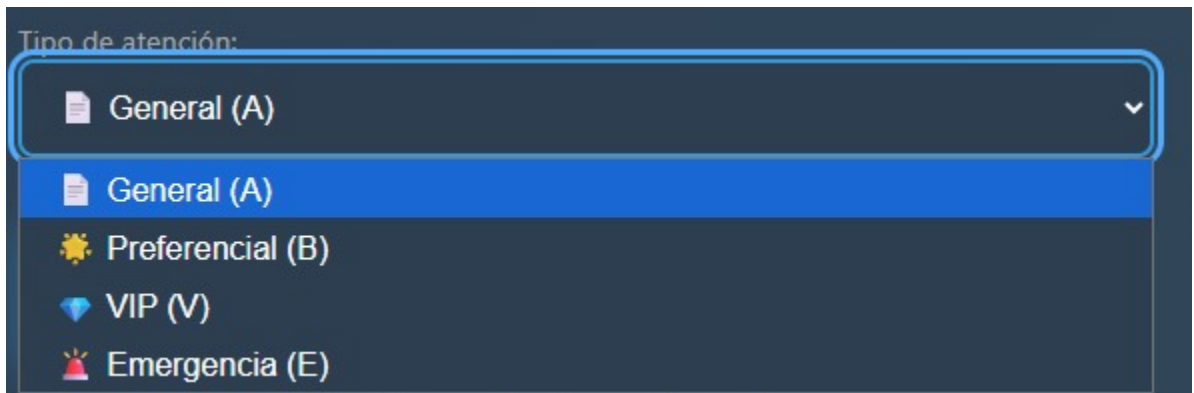
- **FCM:** Utilizado para las **notificaciones push**, asegurando que el cliente reciba la alerta de su turno incluso si no tiene la aplicación abierta en primer plano

Flujo de capturas de funcionamiento

1. ingresar a la app.



2. Ingresar el tipo de atención



3. Genera tu turno.

Tipo de atención:

 Emergencia (E) 

 **PEDIR MI TURNO**

4. Turno agendado. Espera que te llamen 😊



¡Turno Asignado!

Tu número de turno es:

E1

Espera cómodamente. Te notificaremos cuando sea tu turno.

No olvides tu número.

ENTENDIDO

Paso a paso:

1. Ingresa a la app
2. Busca tu prioridad de atención (General A, Preferencial B, Vip V, Emergencia E)
3. Agenda tu turno
4. Espera a tu turno (A1, A2, A3, A4...)

ADMIN

1. Ingresa a la app
2. Abrir el dashboard
3. Mirar que turnos hay pendientes
4. Llamar los turnos
5. Atenderlos por tipo de atención
6. Márcalos como completados
7. Seguir con el otro turno
8. Acabar el día.

Informe Técnico: Sistema de Gestión de Turnos

1. Diccionario de Datos (Firestore)

El sistema integra **Google Firebase Firestore** para la sincronización de datos en tiempo real entre el backend y las interfaces de visualización.

Colección: turns

Almacena el registro histórico y actual de cada turno generado en el sistema.

- **number (String):** Identificador alfanumérico único del turno (ej: "A1").
- **status (String):** Estado operativo del turno (waiting, calling, completed).
- **service_type (String):** Categoría asignada al turno (ej: general, preferential).

Colección: current

Documento utilizado para controlar la pantalla de llamado activo.

- **Documento active -> number (String):** Almacena el número del turno que debe parpadear o sonar en la pantalla principal en ese instante.

2. Catálogo de Endpoints (API REST)

La comunicación se realiza mediante una API basada en **Django Rest Framework**.

Método	Endpoint	Descripción
POST	/api/create/	Crea un nuevo turno con prefijo automático según el tipo de servicio.
POST	/api/call/	Cambia el estado del siguiente turno en espera a "calling".
GET	/api/all/	Retorna la lista de todos los turnos del día con su tiempo de espera calculado.
GET	/api/current/	Obtiene la información del turno que está siendo atendido actualmente.
POST	/api/complete/	Marca un turno específico como finalizado.
POST	/api/reset/	Elimina todos los registros de la base de datos (limpieza de jornada).
GET	/api/statistics/	Procesa métricas de atención, distribución por servicio y tiempos promedio.
GET	/api/export/csv/	Genera un archivo CSV descargable con el histórico de turnos.

3. Arquitectura de WebSockets

Se implementa **Django Channels** para permitir una comunicación bidireccional y de baja latencia.

Configuración de Ruta

- **URL:** ws://[dominio]/ws/turns.
- **Consumer:** TurnConsumer (Async).

Funcionamiento

1. **Conexión:** El cliente se une al grupo turns_updates al conectar.
2. **Acciones del Cliente:** El cliente puede enviar un JSON con la llave action para solicitar datos bajo demanda (get_current, get_all, get_waiting, get_position).
3. **Broadcast (Servidor):** Cada vez que se ejecuta una acción en la API (como crear o llamar un turno), la función broadcast_turn_update envía automáticamente el estado actualizado a todos los clientes conectados vía WebSockets.

Formato de Mensaje (Ejemplo de actualización)

JSON

```
{  
  "type": "current_turn",  
  "number": "A15",  
  "status": "calling"  
}
```

Guía de Instalación y Configuración del Sistema

Este documento detalla los pasos para poner en marcha el servidor de gestión de turnos y la aplicación móvil.

1. Requisitos Previos (Servidor)

La computadora que actuará como servidor debe tener instalados los siguientes componentes:

- **Python 3.12:** Lenguaje base del backend.
- **Pip:** Gestor de paquetes de Python (incluido con Python).
- **Redis:** Necesario para gestionar las capas de canales de WebSockets (Django Channels).
- **Git:** Para la descarga/clonación del código fuente.

2. Variables de Entorno y Credenciales

El sistema requiere credenciales específicas que no están incluidas en el código fuente por seguridad. Estas deben configurarse en un archivo `.env` en la raíz del proyecto o como variables del sistema.

Variable	Origen	Descripción
SECRET_KEY	Django	Llave única para cifrado de sesiones y seguridad.
DEBUG	Config	Debe ser False en producción.
FIREBASE_KEY_PATH	Firebase	Ruta al archivo <code>firebase_key.json</code> (generado en la consola de Firebase).
GEMINI_API_KEY	Google AI	Token para que el asistente de IA funcione (<code>asistente_ia.py</code>).
CLOUDINARY_URL	Cloudinary	Credenciales para el almacenamiento de imágenes (si aplica).

Nota Crítica: El archivo `firebase_key.json` debe colocarse en la carpeta principal del proyecto para que `firebase_config.py` pueda inicializar la base de datos Firestore.

3. Comandos de Arranque

Fase A: Preparación del entorno

1. Crear un entorno virtual: `python -m venv venv`
2. Activar entorno:
 - Windows: `venv\Scripts\activate`

- Linux/Mac: `source venv/bin/activate`
- 3. Instalar dependencias: `pip install -r requirements.txt`
- 4. Aplicar migraciones: `python manage.py migrate`

Fase B: Ejecución del Servidor

Para que el servidor sea accesible desde otros dispositivos en la misma red (como la App móvil o pantallas), ejecute:

Bash

```
python manage.py runserver 0.0.0.0:8000
```

- **Acceso Web:** `http://[IP-DEL-SERVIDOR]:8000/`
 - **Acceso WebSockets:** `ws://[IP-DEL-SERVIDOR]:8000/ws/turns`
-

4. Instalación de la Aplicación Móvil (.APK)

Una vez generado el archivo mediante **EAS (Expo Application Services)**, el cliente debe seguir estos pasos:

1. **Transferencia:** Copiar el archivo .apk al almacenamiento del dispositivo Android.
 2. **Permisos:** Habilitar la opción "Instalar aplicaciones de fuentes desconocidas" en los ajustes de seguridad de Android.
 3. **Ejecución:** Abrir el archivo desde el administrador de archivos y seleccionar "Instalar".
 4. **Configuración:** Al abrir la app, ingresar la dirección IP del servidor configurado en el paso anterior para establecer el enlace.
-

5. Mantenimiento y Limpieza

Para reiniciar el sistema al inicio de una nueva jornada, se puede utilizar el endpoint de reset o el comando:

```
python manage.py reset_turns (si se implementó como comando de gestión) o vía API POST a /api/reset/.
```